FOL: A Functional Object Lisp

Frank Adrian
Ancar Technology
Olathe, KS, USA
frank.adrian314159@gmail.com

Abstract

We present FOL (Functional Object Lisp), a Lisp dialect combining Clojure's persistent data structures with CLOS-style object orientation. Using this combination, we find that persistent objects and CLOS are not antagonistic but, in fact synergistic. Among other software capabilities, the combination yields metaobject-protocol-enabled versioned objects, declarative event sourcing using method combinations, and lazy schema evaluation—patterns more natural than in either parent alone. Benchmarks show that persistence adds only 1–3x overhead for sequential modifications. A preliminary transpiler to Common Lisp with minor optimizations reaches parity with hand-written Common Lisp code. FOL is written in Common Lisp atop FSet and Sycamore persistent data structure libraries, providing Clojure-compatible persistent collection objects and a meta-object-protocol adapted for immutable storage.

CCS Concepts

• **Software and its engineering → Functional languages**; **Object oriented languages**; *Extensible languages*; *Language features.*

Keywords

Functional programming, object-oriented programming, Lisp, persistent data structures, CLOS, MOP, transducers

1 Introduction

The first question to answer when writing a new Lisp is "Why write another Lisp?". In the case of FOL, we wanted to find out if Common Lisp-style objects could co-exist with an immutable object model as found in Clojure. Each tradition offers distinct strengths: CLOS provides multiple dispatch, method combinations, and full metaobject protocol introspection; Clojure provides immutable data structures with $O(\log n)$ updates and structural sharing.

That immutable objects have many advantages in multi-processing systems is a well-known fact. Mutation makes it difficult to coordinate multiple threads without conflict and proper synchronization of these threads across multiple objects is a onerous, labor-intensive, and quite often an ad-hoc, manual task. Systems based on immutable data structures do not require synchronization of this kind.

On the other hand, CLOS, with its multiple-inheritance structures and meta-object protocol (MOP) provides capabilities that other object models do not have. But this object model is fundamentally based on mutation of object instance's variables. Putting CLOS within the paradigm of immutability seemed unlikely; eschewing CLOS's capabilities to gain immutability seemed a high price to pay. In the end, the only way to see if one could merge the two models was to try.

So why build a new Lisp rather than simply building an add-on library using libraries like FSet or Sycamore (both of which are used in FOL)? The first two arguments are due to correctness. FOL, by necessity, has some objects that are not persistent objects—primitives, mutable objects (such as streams, atoms, lazy sequences, etc.), collections objects that are based on persistent data structures, but do not inherit from a persistent-class, as their slots never change and, as such, they don't need to be based on persistent objects. Hooking up these elements properly would have made for a difficult and brittle library system. Another sticking point was that as certain objects required inheritence from CLOS's `standard-object`, if this class were exposed by a library system, there would always be the temptation to use this class as a base for some user class—a land mine of sizable proportion. Guarantees stemming from limiting user-defined objects to be persistent would evaporate, leading to difficult and brittle software. The final argument was pragmatic—it was simply easier to package FOL as a fully-functional Lisp, rather than as a library.

Using Common Lisp's MOP, FOL provides immutable CLOS objects. We show that CLOS objects need not be based on mutable variables, but can instead be based on immutable data structures, where changing variables provide new copies of the original objects sharing structure with the original object. In doing this, we also demonstrate that immutability ensures old instances are never corrupted by class redefinition, and functional updates transparently produce instances of the new schema—trading CLOS's explicit migration hooks for corruption safety.

We demonstrate that the combination of immutability and CLOS-style objects is synergystic. Our paper hgihlights two patterns that are more natural in FOL than in either parent alone:

- **Automatically versioned objects via persistence and the MOP**: Immutability ensures old instances are never corrupted by class redefinition, and functional updates transparently produce instances of the new schema—trading CLOS's explicit migration hooks for corruption safety.
- **Event sourcing via method combinations**: `:around` methods declaratively intercept all mutations to build immutable event logs, while predicate dispatch routes commands by content. FOL's `:around` applies automatically to all methods; Clojure's equivalent wrapper must be invoked explicitly.

FOL also provides enhanced destructuring patterns and predicate dispatch, allowing methods to dispatch on arbitrary predicates in addition to type. This enables more declarative code, such as range-based dispatch or content-based routing, without sacrificing predictability.

Finally, we provide transpiled benchmarks showing that FOL's persistence adds only 1–3x penalty for sequential persistent object

modifications while providing parity with hand-written Common Lisp versions of these same benchmarks.

2 Language Design

2.1 Core Philosophy

FOL makes several design commitments:

(1) **Immutability by default**: All objects and collections (except streams, atoms, etc.) are persistent—in the sense of Driscoll et al. [5], preserving previous versions after modification, not database persistence.
(2) **Structural sharing**: Updates create new versions sharing structure with old.
(3) **Object identity**: Distinguishes version identity (`eq`) from value equality (`=`).
(4) **Generic functions**: Multiple dispatch over destructuring patterns.
(5) **Meta-object protocol**: Introspection and extension via adapted MOP protocols.

2.2 Identity and Equality

Following Hickey's epochal time model [11], FOL simplifies Common Lisp's four-level equality hierarchy (`eq`/`eql`/`equal`/`equalp`) to two levels. `eq` tests *reference identity*—each functional update produces a new object, so `(eq alice older-alice)` returns false even when both represent "Alice." FOL's `=` tests *value equality* via generic dispatch: numbers compare with numeric coercion, persistent objects compare by structural comparison of storage maps (using `fset:equal?`), and strings compare by content—subsuming the roles of `eql`, `equal`, and `equalp`. Two independently constructed objects with identical slot values are `=` and may be used as map keys (hashing is based on storage structure, not reference identity).

2.3 Syntax and Readability

FOL adopts Clojure's reader and definition syntax for literals and destructuring, as these are more concise and readable than Common Lisp's. Persistent collections use literal syntax with Clojure semantics:

```
[1 2 3]                 ; Vectors
{:name "Alice" :age 30}  ; Maps
#{1 2 3 4}              ; Sets

(defclass <person> []
  [[name :type <string>]
   [age :type <number>]])

(defn summarize [{:keys [name age]}]
  (str name " is " age " years old"))
```

2.4 Persistent Object Protocol

By default, user-defined classes inherit from `<persistent-object>`:

```
(defclass <person> []
  [[name :type <string>]
   [age :type <number>]])

(def alice (make <person> :name "Alice" :age 30))
(def older-alice (assoc alice :age 31))  ; Returns new instance
(:age alice)       ; => 30 (unchanged)
(:age older-alice) ; => 31
```

`defclass` supports standard CLOS options (`:type`, `:initarg`, `:initform`, `:reader`). Slot values are stored in a persistent hash map, enabling $O(\log n)$ updates with structural sharing.

3 Architecture and Implementation

3.1 Collection Implementation

FOL wraps FSet [3] and Sycamore data structures for its collections:

- **Vectors** (`[1 2 3]`): 32-way branching trees supporting $O(\log_{32} n)$ random access.
- **Maps** (`{:a 1 :b 2}`): Hash array mapped tries (HAMTs) with efficient key-value operations.
- **Sets** (`#{1 2 3}`): Weight-balanced binary trees for ordered iteration.
- **Lists**: Persistent cons cells with standard $O(1)$ prepend and $O(n)$ access.

FOL also provides thirteen additional collection classes from `<array-dict>` to `<dense-int-set>` to `<array>`. These collection classes have either been optimized to use the combinations of FSet and Sycamore data structures that provide the best performance at both small sizes (< 20 elements) and at scale (> 1000 elements) or have been hand-written in the case of collections not based on persistent structures (list, lazy-seq).

3.2 Runtime Representations

Most runtime representations in the transpiled code other than collections and user-defined objects utilize a Common Lisp base. Primitives (like `<bool>`, `<string>`, the numeric tower, and `<symbol>`) are unwrapped Common Lisp primitives; functions with their closures are simply Common Lisp functions; FOL derives its package structure and error handling model from Common Lisp. Garbage collection is also inherited from Common Lisp—old versions of user-defined objects are collected when unreferenced; structural sharing ensures only unshared portions are reclaimed.

3.3 MOP Protocol Adaptation

FOL's persistent metaclass adapts rather than replaces the MOP. Table 1 summarizes the protocol disposition.

Introspection protocols (`class-slots`, `class-precedence-list`, `slot-definition-*`) work unchanged because FOL's class *structure* is standard CLOS—only slot *storage* is redirected. The `validate-superclass` extension permits cross-metaclass inheritance (persistent classes may inherit from standard classes), but mixed hierarchies must respect the invariant that persistent slots use FSet or Sycamore storage while inherited standard slots remain mutable—finalization signals an error if this invariant cannot be maintained. This adaptation requires the MOP: persistence cannot be implemented as a library because transparent slot access demands `slot-value-using-class` redirection at the metaclass level.

4 Features

4.1 Clojure Features

FOL implements key Clojure abstractions:

Table 1: MOP Protocol Adaptation

Protocol	Status	Rationale
`slot-value-using-class`	Redirected	Reads from FSet map
`(setf slot-value-using-class)`	Guarded	Blocks post-init writes
`slot-boundp-using-class`	Redirected	Checks FSet map membership
`slot-makunbound-using-class`	Blocked	Immutability invariant
`validate-superclass`	Extended	Cross-metaclass mixing
`initialize-instance`	Extended	Populates FSet map from initargs
`update-instance-for-redefined-class`	Omitted	Instances are immutable; lazy migration instead (Sec. 5.4)
`change-class`	Omitted	Violates immutability
Class computation, method combination, allocation: standard CLOS behavior inherited unchanged.		

- **Transducers** [12]: Composable transformation pipelines separating the "what" (map, filter, take) from the "how" (eager, lazy, parallel).
- **Lazy sequences**: On-demand computation enabling infinite sequences and efficient streaming.
- **Thread-safe atoms**: Mutable references with per-atom locking for concurrent updates.
- **Sequence functions**: Over 100 functions including `map`, `filter`, `reduce`, `take`, `drop`, `partition`, `group-by`, and `frequencies`.
- **Macros**: Clojure-style `defmacro` with syntax-quote (`` ` ``), unquote (`~`), and splicing unquote (`~@`); macros receive unevaluated forms. Example: `(defmacro unless [test & body] `(if (not ~test) (do ~@body)))`.
- **Atoms, Refs/Software Transactional Memory, and Agents**: Clojure's concurrency primitives for managing mutable state in a thread-safe manner.
- **Reader syntax**: Clojure-style literals for vectors, maps, sets, keywords, metadata, etc.

4.2 Generic Function Integration

FOL's generic functions dispatch by arity first, then by type predicates (`<vector>?`, `<dict>?`) and specificity within an arity group. Methods at different arities may coexist on the same generic:

```
(defgeneric process [x])
(defmethod process [(x <vector>)] (vec (map process x)))
(defmethod process [(x <dict>)] (update-vals x process))
(defmethod process [(x <number>)] (* x 2))

(process [1 {:a 2 :b 3} [[4]]])  ; => [2 {:a 4 :b 6} [[8]]]

;; Multi-clause methods with predicate dispatch
(defgeneric process ([a (b (< 10))] [a (b <number>) (c <string>)] [a
      b]))
(defmethod process
  ([a (b (< 10))] (* 2 b))
  ([a (b <number>) (c <string>)] c)
  ([a b] (str a b)))
```

Figure 1 gives the formal grammar for FOL's multi-pattern dispatch syntax.

Predicate semantics: The form `(symbol (fn arg0 arg1 ...))` evaluates `(fn symbol arg0 arg1 ...)`—the bound value is inserted as the first argument. This works with functions, macros, or special forms.

`defn` vs `defgeneric`/`defmethod`: `defn` defines a standalone multi-clause function with predicate dispatch and specificity ordering; it does not create a generic function. `defgeneric` declares a generic function; `defmethod` adds methods to it. Only `defmethod` accepts `:before`/`:after`/`:around` qualifiers. Both `defn` and `defmethod` support multi-clause predicate dispatch.

4.2.1 Method Combinations. Multi-clause `defmethod` forms expand to N separate method definitions. The `:before`, `:after`, and `:around` qualifiers apply per-clause; `call-next-method` follows CLOS semantics.

4.3 Predicate Specializers

FOL extends pattern matching with predicate specializers. The syntax `(var (fn arg0 arg1 ...))` applies the predicate to the bound value at runtime.

4.3.1 Predicate Examples. Predicates work with any function—equality, comparison, type tests, or user-defined. In the first example below, all clauses are predicates (same category), so definition order determines dispatch; the second example mixes type predicates and a catch-all, where cross-category specificity applies:

```
;; Range checking with comparison predicates
(defn age-group
  ([(age (< 13))] :child)
  ([(age (< 20))] :teen)
  ([(age (< 65))] :adult)
  ([age] :senior))

;; Type predicates and user-defined predicates
(defn process-value
  ([(x (<number>?))] (* x 2))
  ([(x (<string>?))] (str x x))
  ([(x (<vector>?))] (vec (map process-value x)))
  ([x] x))
```

4.3.2 Predicate Specificity. FOL orders patterns by *constraint strength*—how narrowly a pattern constrains its accepted values. More specific patterns are tested first, ensuring predictable dispatch regardless of definition order:

This ordering is a deliberate heuristic chosen for predictability rather than full logical subsumption, which is undecidable in general [7]. The intuition is that a predicate like `(= 5)` accepts strictly fewer values than `<number>`, which accepts fewer than an untyped destructuring pattern. The boundary between

```
defn           ::= (defn name (single-clause | multi-clause))
defgeneric     ::= (defgeneric name (single-pattern | multi-pattern))
defmethod      ::= (defmethod name qualifier? (single-clause | multi-clause))
fn             ::= (fn (single-clause | multi-clause))
single-clause  ::= pattern body+
multi-clause   ::= (pattern body+)+
single-pattern ::= pattern
multi-pattern  ::= pattern+
pattern        ::= [param* ] | [param* & rest-param]
param          ::= symbol                                  (simple binding)
                 | (symbol type)                           (type specializer)
                 | (symbol (fn-name arg*))                 (predicate specializer)
                 | destructure                             (nested destructuring)
destructure    ::= [param* ]                               (sequential)
                 | {:keys [key-spec* ]}                    (map)
key-spec       ::= symbol | (symbol type) | (symbol pred)
type           ::= <type-name>
qualifier      ::= :before | :after | :around
```

Figure 1: FOL Multi-Pattern Dispatch Grammar (simplified)

Table 2: Predicate Specificity Levels

Level	Pattern Type	Constraint	Example
3	Predicate	Single value/narrow range	`(n (= 5))`
2	Type	All instances of a type	`(x <number>)`
1	Destructuring	Structural shape only	`[a b c]`
0	Any	Universal match	`x`, `_`

type and predicate categories is conventional: type predicates like (`<number>?`) are recognized syntactically and assigned level 2, while other predicates (including user-defined type tests) receive level 3. This means (`x (<number>?)`) and (`x <number>`) accept the same values but occupy different specificity levels if mixed: a clause using (`<number>?`) (level 3) will shadow one using `<number>` (level 2) for numeric arguments, even though both match the same set. In practice this is benign—programmers would use one form or the other, not both—but it is a pragmatic tradeoff we accept for uniform, predictable dispatch. Within a category, definition order is preserved, giving programmers explicit control over tie-breaking. For sequence patterns, specificity extends to element-level comparisons: `[(x (< 0)) y]` is more specific than `[(x <number>) y]` because its first element has higher specificity.

4.3.3 Formal Specificity Ordering. We formalize the specificity relation $\prec$ using inference rules. Let $\mathcal{L}(p) \in \{0, 1, 2, 3\}$ be the specificity level of pattern p.

$$\textsc{Spec-Level}\ \frac{\mathcal{L}(p_1) > \mathcal{L}(p_2)}{p_1 \prec p_2} \qquad \textsc{Spec-Seq-Head}\ \frac{p_1 \prec q_1 \quad n \geq 1}{[p_1 \dots p_n] \prec [q_1 \dots q_n]}$$

$$\textsc{Spec-Seq-Tail}\ \frac{\mathcal{L}(p_1) = \mathcal{L}(q_1) \qquad [p_2 \dots] \prec [q_2 \dots]}{[p_1 \dots p_n] \prec [q_1 \dots q_n]}$$

The operational dispatch order $\prec_{\text{disp}}$ is a total order that resolves ties (where neither $p_1 \prec p_2$ nor $p_2 \prec p_1$) using definition order. Let $idx(p)$ denote the definition index of the method clause containing p.

$$p_1 \prec_{\text{disp}} p_2 \iff p_1 \prec p_2 \vee (\neg(p_2 \prec p_1) \wedge idx(p_1) < idx(p_2))$$

Patterns of different arity are incomparable under $\prec$; dispatch selects the arity group first, then applies $\prec_{\text{disp}}$ within it. This lexicographic combination ensures deterministic dispatch: arity-first, then specificity, then definition-order.

4.3.4 Dispatch Semantics and Type Checking. Predicate specializers also work in anonymous `fn` forms but not in macro parameter lists. Dispatch is $O(N)$ in the number of patterns at a given arity, with patterns pre-sorted by specificity. FOL's type system is entirely dynamic—type specializers are checked at runtime. The compiler has not been extended to warn about unreachable clauses; a future compiler version could detect shadowing statically.

4.3.5 Soundness Properties. FOL's dispatch maintains three invariants: (1) **exhaustive matching**—unmatched calls signal an error `no-matching-clause` rather than silently failing; (2) **deterministic dispatch**—the total order $\prec_{\text{disp}}$ ensures exactly one clause matches; (3) **type predicate consistency**—type specializers use the same predicates as explicit checks. FOL does *not* provide static type soundness; errors are detected at runtime only.

5 Synergy in Practice

This section demonstrates patterns that are *more natural or more concise* in FOL than in either Clojure or standard CLOS alone, because they exploit the combination of persistent data, CLOS method combinations, and predicate dispatch. We focus on patterns that *require* multiple features working together—not merely features used in proximity but features whose interactions produce capabilities unavailable in either parent language. Full source for all examples is available in the repository: FOL code in `fol-code/`,

Common Lisp equivalents in `lisp-code/`, and Clojure equivalents in `clojure-code/`.

5.1 Declarative Dispatch: Trade Compliance

We implement a trade compliance validator in FOL and plain Common Lisp (repository files `compliance.fol`, `compliance.lisp`). The FOL version uses predicate dispatch with automatic specificity ordering; the CL version uses a manual cond cascade with hand-ordered predicates. Table 3 compares the implementations (non-blank, non-comment lines).

```
;;    Inherits from <persistent-object> for immutability
(defclass <trade> [<>]
  [[symbol :initarg :symbol :accessor trade-symbol]
    [amount :initarg :amount :accessor trade-amount]
    [price  :initarg :price  :accessor trade-price]
    [side   :initarg :side   :accessor trade-side]]) ; :buy or :sell

(defn trade-total-price [trade]
  (* (trade-price trade) (trade-amount trade)))

;;    These functions return true/false based on the trade object
(defn restricted-symbol?
    ([(trade (-> (trade-symbol) (#{AMZN GOOG META MSFT})))] t)
    ([trade] nil))

(defn high-value?
    ([(trade (-> (trade-total-price) (> 1000000.00)))] t)
    ([trade] nil))

(defn is-buy-side-trade?
    ([(trade (-> (trade-side) (= :buy)))] t)
    ([trade] nil))

(defn is-penny-stock?
    ([(trade (-> (trade-price) (< 5.00)))] t)
    ([trade] nil))

(defn penny-stock-buy? [trade]
  (and (is-buy-side-trade? trade) (is-penny-stock? trade)))

(defgeneric validate-trade [trade])

;;    Syntax: (var predicate-fn)
(defmethod validate-trade
    ;; Rule 1: Block restricted symbols immediately
    ([(trd (restricted-symbol?))]
        {:status :rejected
          :reason (str "Symbol " (trade-symbol trd) " is on the
      restricted list")})
    ;; Rule 2: Flag high value trades for manual review
    ([(trd (high-value?))]
        {:status :manual-review
          :reason "Trade value exceeds $1M limit"})
    ;; Rule 3: Warn on penny stock purchases
    ([(trd (penny-stock-buy?))]
        {:status :warning
          :reason "High risk penny stock purchase"
          :trade trd})
    ;; Rule 4: Standard approval (Catch-all)
    ([trd]
        {:status :approved
          :id (gensym "TRD")}))
```

Table 3 shows that savings in LOC are modest (17%) because this example exercises predicate dispatch and module syntax but not persistence or method combinations—it demonstrates a single-feature advantage (predicate dispatch) rather than multi-feature synergy. The predicate category is *larger* in FOL because predicate-dispatch patterns require explicit true/nil branches. The advantage here is *correctness*, not brevity: FOL's specificity ordering guarantees that more-constrained rules fire first regardless of definition order. Reordering or adding rules cannot silently change which clause matches—a class of maintenance bug that plagues `cond` cascades in production.

Table 3: Code Comparison—Trade Compliance

Category	FOL	CL	Savings
Class definition	5	8	38%
Predicates / logic	16	13	−23%
Dispatch / validation	15	15	0%
Module boilerplate	2	10	80%
Total	**38**	**46**	**17%**

5.2 Event Sourcing with Method Combinations

FOL enables event sourcing by combining `:around` methods with persistent event logs. The key mechanism is a single `:around` method that intercepts all command dispatches:

```
(defmethod apply-command :around [agg cmd]
  (bind [result (call-next-method)
         event  {:command cmd :timestamp (now)}]
    (assoc result :events
           (conj (:events result) event))))
```

This `:around` method applies automatically to every `apply-command` invocation, including future methods added after deployment. The Clojure equivalent uses `defmulti`/`defmethod` but lacks `:around` methods, so event logging requires a manual wrapper that callers must remember to invoke. Table 4 compares the two (see repository for full source).

Table 4: Code Comparison—Event Sourcing

Category	FOL	Clj	Notes
Class / factory	2	2	Map literal
Event logging	4	4	`:around` vs wrapper
Command methods	7	5	Pred. vs key dispatch
Replay	2	2	Identical
Total	**15**	**14**	

Line counts are nearly identical, but the two implementations differ qualitatively. FOL's `:around` method is *declarative*—it applies automatically to all invocations, including future methods; Clojure's wrapper requires callers to remember to use `apply-command-logged`, and calling the multimethod directly silently skips logging. Persistence contributes a second guarantee: because `assoc` and `conj` produce new immutable values, the event log is append-only by construction—no code path can retroactively alter a recorded event, which is the core invariant event sourcing requires. In mutable CLOS, both the ‘:around‘ interception and defensive copying of the event list would be needed to achieve the same guarantee.

5.3 Observable Slots via MOP and Predicate Dispatch

This example combines persistent functional updates with predicate dispatch to build a change-notification system (repository file `observable.fol`). When a slot is updated, the protocol produces both a new object (via persistent `assoc`) and a change event; predicate dispatch routes notifications by content—e.g., detecting reading spikes (>50 unit jump) or critical status transitions. The CL equivalent (`observable.lisp`) requires a manual copy function, a separate `defstruct` for change events, and a hand-coded `cond` cascade. Table 5 compares the implementations.

```
;; Domain class
(defclass <sensor> []
  [[name     :initarg :name]
   [reading  :initarg :reading  :initform 0]
   [status   :initarg :status   :initform :normal]])

;; A change event is a persistent map produced by the update
    protocol
(defn make-change [obj slot old-val new-val]
  {:object obj :slot slot :old old-val :new new-val})

;; Generic update that produces a change event alongside the new
    object
(defn updated [obj slot new-val]
  (bind [old-val (slot slot obj)
         new-obj (assoc obj slot new-val)]
    {:object new-obj
     :change (make-change obj slot old-val new-val)}))

;; Predicate dispatch routes change notifications by content
(defgeneric on-change [change])

;; Spike detection: reading jumps by more than 50
(defmethod on-change
  ([(c (-> :change (-> :slot (= :reading)))
       (-> :change (-> (fn [ch] (> (- (:new ch) (:old ch)) 50)))))
    ]
   {:alert :spike
    :message (str "Spike detected: "
                  (:old (:change c)) " -> " (:new (:change c))))}))

;; Status transition to :critical
(defmethod on-change
  ([(c (-> :change (-> :slot (= :status)))
       (-> :change (-> :new (= :critical))))]
   {:alert :critical
    :message "Sensor entered critical state"}))

;; Default: log the change
(defmethod on-change
  ([c]
   {:alert :info
    :message (str "Slot " (:slot (:change c))
                  " changed: " (:old (:change c))
                  " -> " (:new (:change c)))}))
```

Table reftab:observable shows a 40% reduction in lines of code. The CL version requires more boilerplate to achieve the same functionality, while FOL's combination of features enables a more concise and declarative implementation.

The 40% reduction comes primarily from eliminating boilerplate: FOL needs no manual copy function (persistent `assoc` shares structure), no separate `defstruct` for change events (persistent maps suffice), and no package ceremony. This example best illustrates three-way synergy: persistence provides safe functional updates that produce change events without corrupting the original; predicate dispatch routes those events by content (e.g., reading spikes, critical transitions); and generic functions with `defmethod` enable new notification rules to be added without modifying the existing dispatch. None of these features alone suffices—Clojure has persistence but no method dispatch for extensible routing; CLOS has dispatch but requires manual deep copies to produce safe change events.

Table 5: Code Comparison—Observable Slots

Category	FOL	CL	Savings
Class / boilerplate	6	23	74%
Update protocol	7	9	22%
Change dispatch	18	20	10%
Total	**31**	**52**	**40%**

5.4 Lazy Schema Evolution

Standard CLOS handles class redefinition by lazily updating existing instances via `update-instance-for-redefined-class`: each instance is mutated in place the next time it is accessed. This protocol assumes mutable instances—an assumption FOL deliberately violates.

FOL's persistent metaclass resolves this tension through what amounts to *lazy schema evolution*. Class definitions remain mutable (redefining a class updates the class object normally), but existing instances are frozen snapshots whose `%persistent-storage` FSet maps are never modified. The MOP's `slot-value-using-class` method provides graceful degradation: when accessing a slot that was added by a redefinition but is absent from an old instance's storage map, the method falls back to evaluating the slot's `:initform` if one exists, or signals `slot-unbound` otherwise. Slots removed by redefinition remain in the storage map but become inaccessible (no slot definition routes to them). Crucially, functional updates perform implicit migration of already existing objects. `set-pslot-value` calls `(class-of object)` to obtain the *current* (redefined) class, allocates a fresh instance of that class, and copies the old storage map forward. The resulting object is an instance of the new schema carrying the old data—new slots resolve via initforms, removed slots persist harmlessly in the map. Persistence makes this *safe* in a specific sense: old instances are never corrupted and remain valid snapshots of the prior schema; in mutable CLOS, redefinition silently mutates every reachable instance. The tradeoff is that CLOS's `update-instance-for-redefined-class` provides an explicit migration hook where the programmer can transform slot values, while FOL's lazy approach silently returns initform defaults for new slots on old instances—a form of graceful degradation that could mask schema mismatches if the programmer is not aware. This lazy migration strategy is, to our knowledge, a novel adaptation of the MOP for immutable object systems—it trades explicit migration control for corruption safety and zero-downtime operation.

6 Evaluation

6.1 Comparison with Related Languages

Table 6: Built-in Feature Comparison

Feature	FOL	Clojure	CL
Persistent objects	✓	✓	×
CLOS/MOP	✓	×	✓
Method combinations	✓	×	✓
Transducers	✓	✓	×
Lazy seqs	✓	✓	×
Predicate dispatch	✓	∼[a]	×[b]
Macros	✓	✓	✓

[a] `defmulti` dispatches on an arbitrary fn but lacks per-parameter specializers and specificity ordering.
[b] EQL specializers only; `optima`/`trivia` provide pattern matching as libraries.

FOL uniquely combines Clojure's functional features with Common Lisp's object system. Persistent structures incur $O(\log n)$ vs $O(1)$ for mutable operations, but high branching factors (32–64) keep constant factors small.

6.2 Benchmark Methodology

All measurements were taken on SBCL 2.6.0 (Windows 11, AMD Ryzen 9 5900X, 64 GB RAM). Performance benchmarks report the mean of 100 runs (coefficient of variation <2%); memory benchmarks report single allocations via `sb-ext:dynamic-space-usage`. We compare against native Common Lisp rather than Clojure: SBCL compiles to native code, providing a strict upper bound on overhead, and since FOL's implementation *is* Common Lisp, this isolates abstraction cost. A Clojure comparison would conflate language design with JVM runtime differences (startup, JIT warmup, GC pauses).

6.3 Memory Benchmarks

6.4 Isolating Persistence from Interpretation

To separate the cost of persistence from the cost of interpretation, we benchmarked FSet operations compiled directly to native code (no FOL interpreter), averaged over 100 runs. Sequential operations (map, reduce) incur 1.1–2.7x overhead; random updates incur 21–22x ($O(\log n)$ vs $O(1)$); dictionary lookups and inserts incur 6–39x. Comparing these figures to the 300–500x overhead in the Map Performance table, we conclude that **interpretation accounts for the vast majority of overhead**, while **persistence adds only 1–3x for sequential operations**. Compilation is therefore the primary optimization target.

6.5 Preliminary Transpiler Results

The unoptimized transpiler already produces code within 1.4x of hand-written CL, reflecting the modest cost of `&rest` dispatch under SBCL's `(speed 3)` optimization. The fixed-arity optimization eliminates this remaining overhead, reaching parity with hand-written code (0.93x—within measurement noise). The transpiled code performs the same logical operations as the hand-written version; the only difference was the dispatch mechanism, which the optimization removes. Comparing against the 300–500x interpreted overhead from Section 6.4 (a different workload), these results confirm that **compilation eliminates dispatch overhead entirely**.

6.6 Threats to Validity

All measurements except the transpiler benchmark (Section 6.6) compare FOL's interpreter against SBCL's native compiler, so reported ratios reflect interpretation overhead, not inherent design costs; the transpiler benchmark compares three SBCL-compiled programs (hand-written, unoptimized transpiled, and optimized transpiled). We do not benchmark against Clojure (see Section 6.2). Memory measurements via `sb-ext:dynamic-space-usage` may include GC artifacts. Code density comparisons use three small, purpose-built examples (38–52 lines); results may differ for larger programs where cross-cutting concerns interact. As a single-author project, FOL has not yet undergone independent code review; the full source is publicly available to facilitate external evaluation.

7 Limitations and Future Work

FOL is primarily interpreter-based; a preliminary transpiler exists (Section 6.6) but covers only a subset of the language. No static analysis exists—type errors and unreachable clauses are detected at runtime. Memory overhead grows with object size (1.1x for 2 slots, 4x for 100 slots). Random updates incur 21x overhead versus $O(1)$ mutation. Class redefinition does not migrate existing instances (Section 5.4); old objects retain their original slot data, with new slots resolved lazily via initforms. Metadata, refs/STM, and agents are not yet implemented.

Planned extensions include **abstract and sealed classes** for exhaustive pattern matching, **parallel collections** following Scala's model, and **channel-based concurrency** via CSP-style go blocks.

8 Related Work

Clojure [10] pioneered persistent data structures in Lisp. Clojure's *protocols* provide type-based polymorphism (single dispatch on the first argument's type) as a lightweight alternative to CLOS generic functions. FOL adopts Clojure's sequence abstraction and transducers but uses CLOS generic functions with multiple dispatch and method combinations—gaining `:before`/`:after`/`:around` qualifiers and MOP introspection at the cost of Clojure's protocol performance optimizations.

Common Lisp's CLOS [2] and MOP [13] provide FOL's object system foundation, extended with persistent slots. **Dylan** [16] was an early implementation of a class-based Scheme. In FOL, it influenced naming conventions (`<type>` notation).

Persistent data structures build on Okasaki [14]; FSet [3] provides production-quality collections for Common Lisp. FOL's functional update pattern is related to **lenses** in the Haskell tradition; FOL does not yet provide a lens abstraction. **Clojure's `defrecord`**, Scala case classes, and Kotlin data classes all provide value-oriented objects, but none integrate with a MOP—FOL is unique in combining persistent storage with metaclass-level extensibility.

8.1 Predicate Dispatch and Pattern Matching

Ernst et al. [7] introduced predicate dispatch with logical implication for specificity. Chambers and Chen [4] demonstrated efficient implementation strategies using dispatch DAGs, achieving performance competitive with hand-written type-case dispatch. **Filtered Dispatch** [15] extended CLOS with predicate-based method selection using explicit priorities. FOL's category-based specificity with first-match tie-breaking provides a middle ground between logical implication (expressive but undecidable) and explicit priorities (simple but error-prone). **Scala's extractors** [6] and **Racket's match expanders** provide extensible pattern matching; **OCaml's polymorphic variants** [9] offer extensible sum types. FOL differs: predicates are arbitrary functions rather than structural extractors, and specificity is determined by category rather than type inclusion.

8.2 Multiple Dispatch and Typed Functional Lisps

Julia [1] uses multiple dispatch but is purely type-based, lacking predicate specializers, method combinations, and MOP introspection. **Coalton** [17] adds Hindley-Milner types to CL—complementary to FOL (static safety vs. persistence). **Typed Racket** [20] adds gradual typing; FOL's type specializers serve as dispatch mechanisms rather than correctness guarantees. **Shen** [19] and **Carp** [18] offer pattern matching and static typing respectively, but lack multiple dispatch with method combinations. **Racket's class system** [8] provides method combinations but separates classes from `match`; FOL unifies pattern matching and method dispatch. **Elixir** combines pattern matching with protocols on the BEAM VM but lacks multiple dispatch and method combinations.

9 Conclusion

FOL demonstrates that persistent objects and CLOS are synergistic rather than competing paradigms. Their combination yields versioned objects via persistent snapshots, declarative event sourcing via method combinations over persistent logs, and observable slots via predicate dispatch over change events—patterns that require multiple features working together and are more natural than in either parent language alone. Lazy schema evolution (Section 5.4) further illustrates how immutability transforms a traditionally problematic MOP protocol into a corruption-safe migration strategy, albeit at the cost of CLOS's explicit migration hooks. Benchmarks confirm that interpretation, not persistence, dominates current overhead (300–500x vs 1–3x). A preliminary transpiler to Common Lisp with a single optimization—fixed-arity dispatch—reaches parity with hand-written code (0.93x) on the compliance benchmark, eliminating the dispatch overhead entirely. This demonstrates that compilation, not language design, is the sole performance bottleneck, and that a complete compiler can retain immutability's correctness and concurrency benefits at native speed. The complete implementation—385 functions, 6,333 tests, and ~18,000 lines of bootstrap code—is available at: https://github.com/frankadrian314159/fol

References

[1] Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B Shah. 2017. Julia: A Fresh Approach to Numerical Computing. *SIAM Rev.* 59, 1 (2017), 65–98.

[2] Daniel G Bobrow, Linda G DeMichiel, Richard P Gabriel, Sonya E Keene, Gregor Kiczales, and David A Moon. 1988. Common Lisp Object System Specification. *SIGPLAN Notices* 23, SI (1988), 1–142.

[3] Scott L Burke. 2007. FSet: A functional set-theoretic collections library. https://common-lisp.net/project/fset/

[4] Craig Chambers and Weimin Chen. 1999. Efficient Multiple and Predicate Dispatching. In *Proceedings of the 14th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*. 238–255.

[5] James R Driscoll, Neil Sarnak, Daniel D Sleator, and Robert E Tarjan. 1989. Making Data Structures Persistent. *J. Comput. System Sci.* 38, 1 (1989), 86–124.

[6] Burak Emir, Martin Odersky, and John Williams. 2007. Matching Objects with Patterns. In *ECOOP 2007–Object-Oriented Programming*. Springer, 273–298.

[7] Michael D Ernst, Craig Kaplan, and Craig Chambers. 1998. Predicate Dispatching: A Unified Theory of Dispatch. In *ECOOP'98—Object-Oriented Programming*. Springer, 186–211.

[8] Matthew Flatt, Robert Bruce Findler, and Matthias Felleisen. 2006. Scheme with Classes, Mixins, and Traits. In *Asian Symposium on Programming Languages and Systems*. Springer, 270–289.

[9] Jacques Garrigue. 1998. Programming with Polymorphic Variants. In *ML Workshop*.

[10] Rich Hickey. 2008. The Clojure programming language. In *Proceedings of the 2008 symposium on Dynamic languages*. 1.

[11] Rich Hickey. 2009. Are We There Yet? JVM Languages Summit Keynote. https://github.com/matthiasn/talk-transcripts/blob/master/Hickey_Rich/AreWeThereYet.md

[12] Rich Hickey. 2014. Transducers. Clojure Blog. https://clojure.org/reference/transducers

[13] Gregor Kiczales, Jim Des Rivieres, and Daniel Gureasko Bobrow. 1991. *The Art of the Metaobject Protocol*. MIT press.

[14] Chris Okasaki. 1998. *Purely Functional Data Structures*. Cambridge University Press.

[15] Doug Orleans. 2002. Filtered Dispatch. In *Proceedings of the 17th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*. 20–26.

[16] Andrew Shalit. 1996. *The Dylan Reference Manual*. Addison-Wesley Longman Publishing Co., Inc.

[17] Robert Smith and Cole Bates. 2023. Coalton: Efficient, Statically Typed Functional Programming on Common Lisp. In *Proceedings of the 16th European Lisp Symposium*.

[18] Erik Svedäng. 2018. Carp: A Statically Typed Lisp Without a GC. https://github.com/carp-lang/Carp

[19] Mark Tarver. 2011. *The Shen Language*. Upaya Books.

[20] Sam Tobin-Hochstadt and Matthias Felleisen. 2008. The Design and Implementation of Typed Scheme. In *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. 395–406.